

Unidad 0



La base de la programación

Programación



Índice

Programación | UNIDAD 0
La base de la programación



Conceptos básicos de la programación

Los lenguajes de programación

Lenguaje de aprendizaje

Ejemplos de lenguajes de programación actuales

Requisitos para elegir un lenguaje de programación

El lenguaje de programación JAVA

Características

Primeros pasos con Java



Conceptos básicos de la programación

La programación es una disciplina fundamental en el mundo actual, ya que permite que las personas se comuniquen con las máquinas para indicarles qué deben hacer. Esta capacidad de dar instrucciones a los ordenadores no es algo que haya existido desde siempre, sino que es el resultado de un largo proceso de evolución tecnológica e intelectual.

La idea de dar instrucciones a una máquina tiene raíces mucho más antiguas de lo que podríamos imaginar. Ya en el siglo IX, el matemático persa Al-Juarismi sentó las bases de lo que hoy conocemos como "algoritmos". De hecho, su nombre dio origen al término. Un algoritmo, en términos simples, es una serie de pasos ordenados y finitos que permiten resolver un problema o alcanzar un objetivo. Por ejemplo, una receta de cocina es un algoritmo: sigue una secuencia clara de pasos para lograr un resultado específico. En la programación, un algoritmo cumple la misma función, pero adaptado a resolver problemas mediante instrucciones que una máquina pueda entender.

Durante siglos, los algoritmos se aplicaron de manera manual, es decir, eran las personas quienes los ejecutaban, como en el caso de cálculos matemáticos. Sin embargo, con la Revolución Industrial y más tarde con el desarrollo de la informática, surgió la posibilidad de que las máquinas hicieran este trabajo por nosotros. Una figura muy importante en esta transición fue Ada Lovelace, considerada la primera programadora de la historia. En el siglo XIX, trabajó junto a Charles Babbage en el diseño de una máquina analítica, y escribió lo que se considera el primer algoritmo destinado a ser procesado por una máquina.

Ya en el siglo XX, con el desarrollo de los primeros ordenadores electrónicos, la programación adquirió una nueva dimensión. Se hizo necesario crear lenguajes que permitieran a las personas escribir instrucciones de forma comprensible tanto para ellas como para las máquinas. Así nacieron los lenguajes de programación.

En este punto, es importante entender qué es una instrucción. Una instrucción es una orden concreta que se le da al ordenador para que realice una acción determinada. Por ejemplo, mostrar un mensaje en pantalla, sumar dos números o guardar una información en la memoria. Cuando se juntan varias instrucciones, siguiendo un orden lógico y estructurado, se forma un programa. Un programa no es más que un conjunto organizado de instrucciones que permiten a un ordenador realizar una tarea específica.

Podemos imaginar un programa como una historia que sigue una trama. Cada línea del programa es una instrucción que hace avanzar la acción. Si las instrucciones están bien escritas y ordenadas correctamente, el programa funcionará y cumplirá su propósito. Si, por el contrario, hay errores en las instrucciones o en el orden, el programa puede fallar o comportarse de forma inesperada.



Los lenguajes de programación

En la comunicación humana, utilizamos un lenguaje para transmitir información. Este proceso implica la presencia de un emisor que emite el mensaje, un receptor que lo recibe y un canal a través del cual el mensaje es enviado. Cada idioma que hablamos tiene sus propias reglas gramaticales, vocabulario y estructura, y, aunque estos idiomas pueden variar enormemente, todos comparten una característica fundamental: una sintaxis que regula la forma en que las palabras deben ser pronunciadas o escritas para ser comprendidas correctamente. Este sistema de reglas es necesario para que la comunicación sea efectiva, si no, sería muy difícil establecer un marco de comunicación estable.

En el ámbito de la informática, el concepto de lenguaje también se aplica por igual pero, en lugar de seres humanos como receptores, la máquina es la que recibe y procesa la información. Las computadoras, en su forma más básica, solo pueden interpretar señales eléctricas, que se manifiestan como secuencias de 0 y 1, conocidos como bits. Estos bits representan los dos estados posibles de un transistor, un componente electrónico en los circuitos de una computadora: un estado "abierto" (1) o "cerrado" (0).

Como los seres humanos no podemos interactuar directamente con las máquinas a través de estos impulsos eléctricos, es necesario crear un lenguaje intermedio que sirva como puente entre las intenciones del programador y las operaciones realizadas por la máquina. Este lenguaje se llama "lenguaje de programación", y es a través de él que podemos "traducir" nuestras instrucciones en acciones que la computadora pueda entender.

En los primeros días de la informática, la interacción con las máquinas era extremadamente rudimentaria. Los primeros métodos de programación requerían manipular físicamente el hardware, como en el caso de las tarjetas perforadas, que eran utilizadas para introducir instrucciones. Estas tarjetas contenían agujeros que representaban diferentes instrucciones en código binario, permitiendo que el flujo de electricidad a través de los circuitos se viera alterado de acuerdo con las operaciones deseadas. Aunque eficaz en su época, este método era tedioso y propenso a errores.

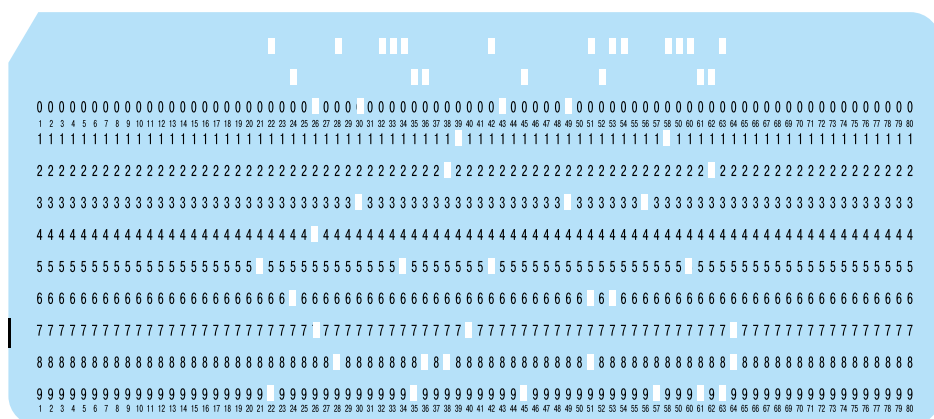


Imagen 1. Tarjeta perforada



Con el tiempo, se desarrollaron lenguajes más avanzados, como el **lenguaje ensamblador**, que ofreció una forma más accesible de escribir instrucciones para las computadoras. El ensamblador se compone de un conjunto de instrucciones que están directamente relacionadas con el hardware de la máquina. Cada instrucción corresponde a una operación elemental que la CPU puede ejecutar, como cargar un valor en un registro o realizar una operación aritmética. A pesar de que sigue siendo un lenguaje de bajo nivel, es mucho más fácil de comprender y escribir que manipular directamente los componentes físicos de la máquina. Sin embargo, el lenguaje ensamblador sigue siendo más cercano al lenguaje de la máquina y requiere un conocimiento detallado de la arquitectura de la computadora.

```
section .data
; Definimos los dos números a sumar
num1 db 5 ; Número 1 (5 en decimal)
num2 db 10 ; Número 2 (10 en decimal)

section .text
global _start

_start:
; Cargamos los números en los registros
mov al, [num1] ; Cargamos el valor de num1 en el registro AL
add al, [num2] ; Sumamos el valor de num2 al registro AL

; El resultado de la suma está ahora en el registro AL
; (AL es un registro de 8 bits en la arquitectura x86)

; Para finalizar el programa
mov ebx, 0 ; Código de salida 0
mov eax, 1 ; Número de la llamada al sistema para salir
int 0x80 ; Interrupción para hacer la llamada al sistema
```

Imagen 2. Código ensamblador para hacer una suma 5 + 10

Cuando observamos el código de ensamblador, como el que aparece en la imagen, vemos una mezcla de letras y dígitos. A simple vista, puede parecer que es posible interpretarlo y entenderlo en nuestro lenguaje. Sin embargo, la computadora no entiende directamente expresiones como “num1”, “section”, “add”, “int”, ... Si tomamos, por ejemplo, la instrucción *mov al, [num1]* y la ejecutamos, es decir, si enviamos esa acción a la computadora, esta comenzará a traducir cada parte de la instrucción a código binario, que es el único formato que entiende.

Supongamos que la computadora tiene una arquitectura de 32 bits, lo que significa que el procesador puede procesar 32 bits por cada acción. La instrucción *mov al, [num1]* se interpretaría de la siguiente manera:

Desglose de la instrucción:

1. **mov**: es la instrucción que indica que se va a **mover o cargar un valor** de una ubicación a otra.
2. **al**: es un registro de 8 bits en la CPU. En este caso, es el **registro al que se moverá el valor** de memoria.
3. **[num1]**: indica que se accede a una **dirección de memoria** en lugar de un valor inmediato. **Los corchetes indican que se trata de una referencia** a una dirección de memoria donde está almacenado el valor de num1.



Traducción de cada parte:

1. La instrucción mov se traduce a un **opcode (código de operación)**, que es un valor de 8 bits. En una arquitectura x86, el opcode para mov es 8A en hexadecimal.
2. El registro a1 es de 8 bits, y la CPU necesita especificar cómo acceder a este registro. Dado que en este caso se accede a una dirección de memoria, el byte que se utiliza es 05, lo que indica que se trata de una dirección de memoria de 32 bits.
3. Supongamos que la dirección de memoria de num1 es 0x1000, lo que representa una dirección de memoria de 32 bits. En binario, 0x1000 se traduce como 0000000100000000.

La instrucción completa se traduce a los siguientes valores en hexadecimal:

8A 05 00 10 00 00

Y, si la convertimos a binario, se vería de la siguiente manera:

10001010 00000101 00000000 00010000 00000000 00000000

Fíjate en la cantidad de 0s y 1s que resultan al traducir solo estas tres partes de la instrucción. En este caso, la secuencia es de **48 bits (6 bytes)** para representar esta única operación. Ahora, si pensamos en un programa más complejo con muchas instrucciones en ensamblador, la cantidad de 0s y 1s que se generarían sería mucho mayor.

La pregunta es: si tradujéramos todo el código ensamblador de un programa completo, ¿cuántos 0s y 1s serían necesarios?

Con el paso de los años, los lenguajes de programación han evolucionado significativamente, pasando de ser herramientas orientadas a la manipulación directa de hardware a lenguajes más abstractos, como **C, Python o Java. Estos lenguajes permiten a los programadores centrarse en la lógica** y los algoritmos sin tener que preocuparse tanto por los detalles del hardware subyacente. A través de estas herramientas, los desarrolladores pueden escribir código que es mucho más legible, mantenible y eficiente, lo que permite un desarrollo de software más rápido y accesible.

Cada uno de estos lenguajes modernos tiene su propia sintaxis y reglas, pero todos siguen el principio fundamental de permitir la comunicación entre el ser humano y la máquina.



Lenguaje de aprendizaje

Cuando se comienza a aprender a programar o se realiza un diseño previo antes de escribir código, el objetivo es estructurar y planificar de manera lógica las soluciones a los problemas sin tener que lidiar inicialmente con la sintaxis de un lenguaje de programación específico.

En esta etapa, no se utilizan los lenguajes de programación directamente, ya que estos requieren una comprensión precisa de su sintaxis, reglas y estructuras. En lugar de eso, se recurre a un lenguaje que se asemeja más a nuestro lenguaje natural, utilizando palabras y expresiones comprensibles para los humanos, pero sin las restricciones estrictas de la sintaxis de un lenguaje de programación.

Este lenguaje intermedio, conocido como **pseudocódigo**, permite describir de manera clara y concisa la lógica de un programa sin preocuparse por los detalles de implementación. El pseudocódigo **no tiene una traducción directa a código binario**, ya que no se ejecuta por sí mismo en una computadora. En cambio, actúa como un puente entre la idea o algoritmo que queremos implementar y el lenguaje de programación específico que utilizaríamos para escribir el código real.

Posteriormente, el pseudocódigo se convierte en un lenguaje de programación mediante un proceso de traducción, donde se adaptan las instrucciones del pseudocódigo a la sintaxis y estructuras del lenguaje elegido, para que la computadora pueda ejecutarlo de manera efectiva.

En el siguiente ejemplo, se muestra el código en pseudocódigo para realizar la suma de dos números.

```
Inicio
    Definir num1 como Entero
    Definir num2 como Entero
    Definir suma como Entero
    Asignar 5 a num1
    Asignar 10 a num2

    suma ← num1 + num2
    Mostrar "La suma de ", num1, " y ", num2, " es: ", suma
Fin
```



Ejemplos de lenguajes de programación actuales

C#

Lenguaje de programación desarrollado por Microsoft, orientado a objetos y utilizado principalmente en el desarrollo de aplicaciones de escritorio, videojuegos con Unity y software empresarial en el ecosistema .NET.

```
using System;

class Program {
    static void Main() {
        int num1 = 5, num2 = 10;
        int sum = num1 + num2;
        Console.WriteLine("La suma de {0} y {1} es: {2}", num1, num2, sum);
    }
}
```

Java

Lenguaje de propósito general, orientado a objetos y ampliamente utilizado en aplicaciones empresariales, desarrollo móvil (Android) y sistemas distribuidos. Se caracteriza por su portabilidad gracias a la máquina virtual de Java (JVM).

```
public class Suma {
    public static void main(String[] args) {
        int num1 = 5, num2 = 10;
        int sum = num1 + num2;
        System.out.println("La suma de " + num1 + " y " + num2 + " es: " + sum);
    }
}
```





Python

Lenguaje de alto nivel, fácil de aprender y con una sintaxis clara. Se usa en desarrollo web, ciencia de datos, inteligencia artificial, automatización y más. Su versatilidad y extensas bibliotecas lo hacen muy popular.

```
num1 = 5
num2 = 10
sum = num1 + num2
print(f"La suma de {num1} y {num2} es: {sum}")
```

Javascript

Lenguaje de programación esencial para el desarrollo web, utilizado principalmente en el lado del cliente para crear interfaces dinámicas. Con Node.js, también se usa en el backend.

```
let num1 = 5;
let num2 = 10;
let suma = num1 + num2;
console.log("La suma de", num1, "y", num2, "es:", suma);
```

PHP

Lenguaje de scripting ampliamente utilizado en desarrollo web para la creación de sitios y aplicaciones dinámicas. Es el motor de muchas plataformas como WordPress y Laravel.

```
<?php

$num1 = 5;
$num2 = 10;
$suma = $num1 + $num2;

echo "La suma de $num1 y $num2 es: $suma";

?>
```



Requisitos para elegir un lenguaje de programación

Al momento de elegir un lenguaje de programación, es necesario considerar una serie de requisitos que permitan tomar una decisión adecuada según el contexto y las necesidades del proyecto. No existe un lenguaje universalmente mejor que otro, sino que cada uno tiene sus propias características, ventajas y desventajas que lo hacen más o menos apropiado para determinadas situaciones.

Uno de los aspectos para tener en cuenta es el **propósito del desarrollo**. No es lo mismo programar un sistema embebido de tiempo real que una aplicación web interactiva o una herramienta de análisis de datos. Algunos lenguajes están diseñados específicamente para ciertos ámbitos, como **JavaScript en el desarrollo web, Python en el análisis de datos o C en programación de sistemas**.

Otro factor clave es el **rendimiento y eficiencia** del lenguaje. Si el sistema requiere un alto rendimiento, como en videojuegos o aplicaciones que procesan grandes volúmenes de datos en tiempo real, puede ser necesario elegir un lenguaje de bajo nivel, como **C o Rust**. En cambio, si la prioridad es la facilidad de desarrollo y la rapidez de implementación, lenguajes como **Python o Ruby** pueden ser una mejor opción.

La **facilidad de aprendizaje y uso** también influye en la elección. Lenguajes con una sintaxis clara y sencilla, como Python, suelen ser preferidos por principiantes o equipos que buscan reducir la curva de aprendizaje. Por otro lado, algunos lenguajes más complejos pueden ofrecer un mayor control sobre el sistema, aunque requieran más esfuerzo para dominarlos.

Un lenguaje con una **gran comunidad** de desarrolladores y **un amplio ecosistema** de bibliotecas y herramientas facilita el desarrollo, permite encontrar soluciones a problemas más rápidamente y garantiza que el lenguaje se mantenga actualizado con el tiempo. Python y JavaScript, por ejemplo, cuentan con comunidades activas que proporcionan constantes actualizaciones y mejoras.

También es importante considerar la **compatibilidad y portabilidad**. Si el sistema debe ejecutarse en múltiples plataformas, se debe optar por un lenguaje multiplataforma, como Java o Python. En cambio, si se trabaja con un ecosistema específico, como aplicaciones para iOS, puede ser más conveniente utilizar Swift.

Otro aspecto es el **soporte y mantenimiento a largo plazo**. Si el proyecto requiere estabilidad y soporte a largo plazo, conviene elegir un lenguaje consolidado con actualizaciones frecuentes y soporte empresarial. Java y C# son ejemplos de lenguajes utilizados en grandes empresas por su estabilidad y soporte continuo.

A parte, se deben evaluar los **costes asociados**. Algunos lenguajes requieren herramientas o entornos de desarrollo de pago, mientras que otros son completamente gratuitos y de código abierto. Además, la disponibilidad de programadores capacitados en un lenguaje puede influir en los costes de contratación y mantenimiento del software.



El lenguaje de programación JAVA

Java es un lenguaje de programación que fue creado por la empresa Sun Microsystems en el año 1995, y desde entonces se ha convertido en uno de los lenguajes más utilizados en todo el mundo. Su creador principal fue James Gosling, y uno de los objetivos desde el principio fue desarrollar un lenguaje que pudiera funcionar en cualquier tipo de dispositivo, sin importar el sistema operativo. De ahí nace uno de sus lemas más conocidos: **“escribe una vez, ejecuta en cualquier lugar”**. Esto significa que un programa escrito en Java puede funcionar tanto en un ordenador con Windows como en un Mac, un servidor o incluso en algunos dispositivos móviles, sin necesidad de modificar el código.

Según el índice TIOBE, Java ocupa un cuarto lugar de los lenguajes más actualizados en el año 2025.

May 2025	May 2024	Change	Programming Language	Ratings	Change
1	1		 Python	25.35%	+9.02%
2	3	▲	 C++	9.94%	+0.41%
3	2	▼	 C	9.71%	-0.27%
4	4		 Java	9.31%	+0.62%
5	5		 C#	4.22%	-2.27%
6	6		 JavaScript	3.68%	+0.66%

Imagen 3. Estadística de TIOBE de lenguajes de programación más usados en 2025

¿SABÍAS QUÉ?

Originalmente el lenguaje se llamaba Oak, pero ese nombre ya estaba registrado. Sus creadores, buscando un nuevo nombre, se inspiraron en el café que bebían mientras trabajaban. Así nació Java, en referencia al café de la isla de Java en Indonesia. Por eso, su logotipo también es una taza de café humeante.

Java se caracteriza por ser un lenguaje de propósito general, lo que significa que se puede usar para hacer casi cualquier tipo de programa: desde aplicaciones de escritorio, hasta páginas web, videojuegos, programas para móviles, sistemas de gestión de empresas o incluso aplicaciones para cajeros automáticos. Además, Java es un lenguaje orientado a objetos, lo que quiere decir que su estructura está pensada para organizar los programas en partes pequeñas y reutilizables llamadas “objetos”, que representan elementos del mundo real o del sistema que estamos creando. Esta forma de trabajar ayuda a mantener el código ordenado, claro y fácil de modificar.

Una de las razones por las cuales **Java es una buena elección para aprender a programar** es su equilibrio entre simplicidad y potencia. Por un lado, su sintaxis (la manera en la que se escriben las instrucciones) es clara y parecida a la de otros lenguajes populares como C++, C# o incluso JavaScript, lo cual facilita el aprendizaje posterior de otros lenguajes.



Por otro lado, Java cuenta con muchas herramientas, bibliotecas y recursos disponibles que permiten crear programas complejos de forma más sencilla, además de tener una comunidad de usuarios enorme que ofrece tutoriales, foros y ejemplos prácticos para resolver dudas.

Comparado con otros lenguajes, Java se sitúa en un punto medio. Por ejemplo, si lo comparamos con Python, podríamos decir que Python tiene una sintaxis más simple y es más directo, por lo que a veces se elige primer lenguaje por su facilidad inicial. Sin embargo, Java obliga a ser más riguroso con la estructura del programa desde el principio, lo cual puede ser una ventaja a largo plazo, ya que entrena al futuro programador/a a pensar de forma organizada y profesional.

En comparación con C++, Java es más seguro y más fácil de manejar en cuanto a errores, ya que gestiona automáticamente aspectos complejos como la memoria del ordenador. Y frente a JavaScript, que se usa sobre todo en páginas web, Java es más versátil para desarrollar programas completos en distintos entornos.

Características

La siguiente tabla hace un resumen de las principales características que posee Java.

Característica	Descripción
Lenguaje orientado a objetos	Es un lenguaje que organiza los programas en clases y objetos, lo que facilita la reutilización del código y una estructura más clara y modular.
Multiplataforma	Se puede ejecutar en distintos sistemas operativos (Windows, Linux, macOS, etc.) sin cambiar el código.
Sintaxis estructurada	Obliga a seguir una estructura clara (uso de clases, métodos, tipos de datos), lo que ayuda a aprender buenas prácticas desde el principio.
Tipado fuerte	Cada variable debe tener un tipo definido (como entero, texto, decimal), lo que reduce errores y mejora la claridad del código.
Lenguaje compilado e interpretado	El código se compila a un formato intermedio llamado bytecode, que luego es interpretado por la Máquina Virtual de Java (JVM).
Gestión automática de memoria	El propio lenguaje incluye un recolector de basura (garbage collector) que libera la memoria que ya no se necesita, ayudando a evitar errores complicados.
Seguridad	Incluye mecanismos para proteger el código, especialmente en aplicaciones que se ejecutan en red o en entornos controlados.
Portable	Al no depender de una plataforma específica, el código puede moverse de un dispositivo a otro con facilidad, siempre que tenga una JVM instalada.

Primeros pasos con Java

Comenzar a programar en Java puede parecer un reto al principio, pero si damos los pasos adecuados y entendemos lo que ocurre en cada momento, pronto veremos que no es tan complicado como parece. Java es un lenguaje muy estructurado, lo que significa que debemos seguir ciertas reglas para que el ordenador entienda lo que queremos hacer. Pero una vez aprendidas esas reglas básicas, podremos construir desde pequeños programas hasta aplicaciones complejas.

Antes de poder escribir y ejecutar nuestros primeros programas en Java, es fundamental preparar correctamente el entorno de trabajo. Aunque Java es un lenguaje muy potente y versátil, necesita de ciertas herramientas instaladas en nuestro ordenador para funcionar correctamente. No basta con abrir un bloc de notas y comenzar a escribir. Java requiere un entorno que le permita traducir y ejecutar nuestras instrucciones.

Preparar el entorno para programar

Lo primero que debemos instalar es el JDK (Java Development Kit). Este kit de desarrollo es el corazón de Java: incluye todas las herramientas necesarias para compilar (traducir) nuestro código y ejecutarlo. Sin el JDK, simplemente no podríamos hacer que nuestros programas funcionen. Existen distintas versiones del JDK, pero lo más recomendable para empezar es descargar la versión oficial desde el sitio web de Oracle o utilizar distribuciones libres como OpenJDK.

Una vez que tenemos instalado el JDK, el siguiente paso es elegir un entorno de desarrollo o IDE (Integrated Development Environment). Un IDE es una aplicación que nos facilita muchísimo la tarea de programar. Nos permite escribir el código con ayuda (por ejemplo, nos sugiere palabras clave o nos avisa de errores), organizar nuestros archivos, compilar el programa con un solo clic y ver los resultados de manera rápida.

Algunos de los IDE más utilizados para programar en Java son:

- > **Eclipse:** muy completo y ampliamente utilizado en el mundo profesional.

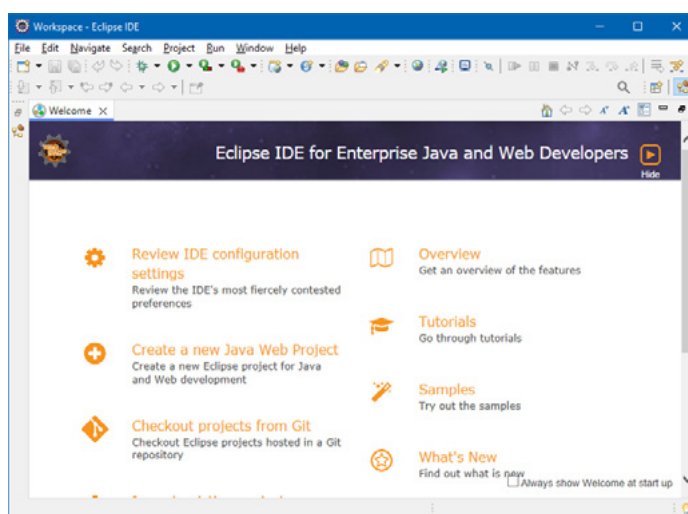


Imagen 4. Entorno de desarrollo Eclipse

- > **NetBeans:** desarrollado originalmente por Sun Microsystems, es sencillo y muy didáctico.

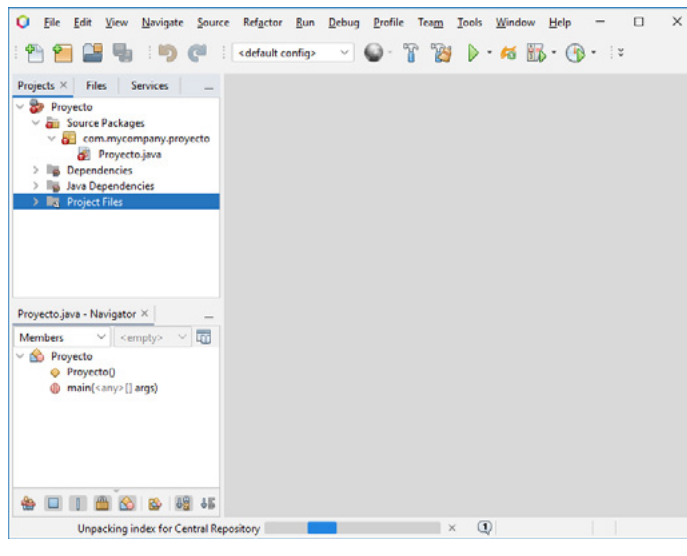


Imagen 5. Entorno de desarrollo NetBeans

- > **IntelliJ IDEA:** muy moderno y potente, especialmente usado en entornos profesionales.

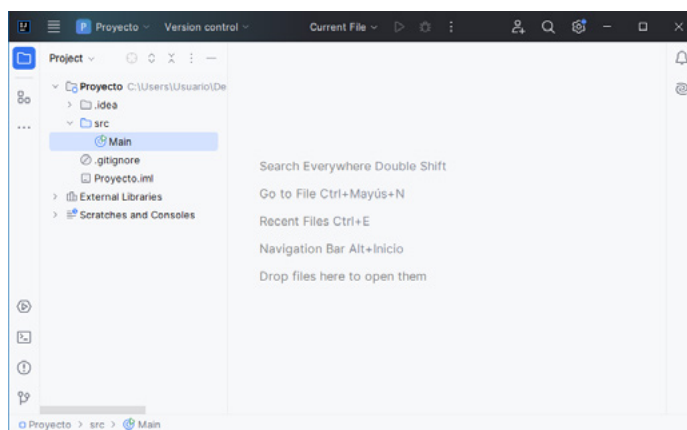


Imagen 6. Entorno de desarrollo IntelliJ IDEA

- > **Visual Studio Code:** no es un IDE puro, pero permite programar en Java instalando algunas extensiones.

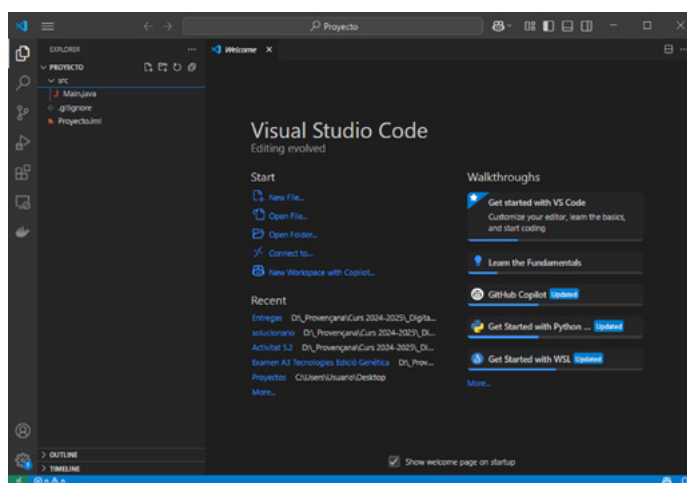


Imagen 7. Entorno de desarrollo de Visual Studio Code



Para quienes están comenzando y aún se sienten un poco inseguros, existe una excelente alternativa llamada BlueJ. Este entorno fue creado específicamente con fines educativos. Su interfaz es muy amigable, y está diseñada para ayudar a entender cómo se estructuran los programas en Java sin distraer al alumno con herramientas complejas. BlueJ permite visualizar de forma clara las clases, objetos y sus relaciones, lo que resulta muy útil en las primeras etapas del aprendizaje.

Una vez instalado todo, el primer objetivo es escribir un primer programa. Tradicionalmente, en programación se comienza con el famoso "Hola mundo", un programa muy simple que simplemente muestra ese mensaje por pantalla. Aunque parece algo muy básico, es muy útil porque nos permite entender cómo se estructura un programa en Java.

Aquí tienes un ejemplo del código de "Hola mundo" en Java:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

Vamos a analizarlo paso a paso. La primera línea, `public class HolaMundo`, indica que estamos creando una clase, que es como una caja donde vamos a meter todo el contenido de nuestro programa. En Java, todo el código debe estar dentro de una clase. El nombre de la clase (en este caso, `HolaMundo`) debe coincidir con el nombre del archivo que guardamos, que será `HolaMundo.java`.

La segunda línea, `public static void main(String[] args)`, define el método principal, también llamado `main`. Este método es el punto de entrada del programa, es decir, es lo primero que se ejecuta cuando el programa arranca. Es como la puerta por la que empieza a correr todo el código.

Dentro del `main`, encontramos la línea `System.out.println("Hola mundo");`. Esta es la instrucción que muestra el mensaje. Le estamos diciendo al sistema (`System`) que saque algo por pantalla (`out.println`), y lo que va entre comillas es el texto que queremos que se vea.

Aunque este ejemplo es muy pequeño, ya estamos trabajando con muchos de los elementos esenciales de la programación en Java: clases, métodos, instrucciones y mensajes. Es importante entender que Java requiere que respetemos ciertas normas, como el uso de las llaves `{}` para agrupar instrucciones, el punto y coma al final de cada línea, y la necesidad de seguir una sintaxis precisa. Estos detalles pueden parecer molestos al principio, pero son fundamentales para que el programa funcione correctamente.

Una vez escrito el código, debemos **compilarlo**. Eso significa que el código que hemos escrito (en lenguaje que entiende una persona) se traduce a un código intermedio (bytecode) que puede ser ejecutado por la máquina virtual de Java (JVM). Si no hay errores, se creará un archivo llamado `HolaMundo.class`, y luego podremos ejecutarlo para ver el mensaje en pantalla.

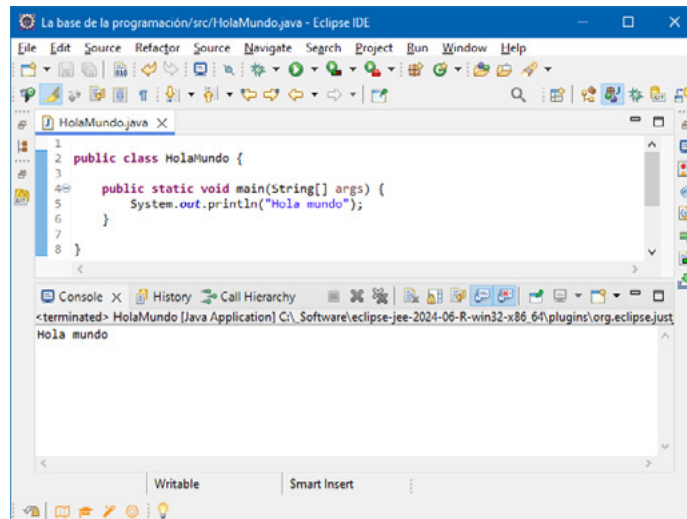


Imagen 8. Ejecución del código Hola mundo en el IDE Eclipse





 www.universae.com

